

Winventory - Gestion Intelligente des Stocks de Boissons

Table des matières

-  [Équipe](#)
-  [Description du Projet](#)
-  [Fonctionnalités Principales](#)
-  [Objectif](#)
-  [Compilation, édition et lancement du projet](#)
-  [Documentation de l'API](#)
-  [Utilisation de l'application web avec cURL](#)
-  [Installation et Configuration d'une Machine Virtuelle](#)
-  [Configuration de la zone DNS pour accéder à l'application web avec DuckDNS](#)
-  [Détails de l'API](#)

Équipe

- **Lestiboudois Maxime**
- **Parisod Nathan**
- **Surbeck Léon**

Description du Projet

Winventory est une application web conçue pour faciliter la gestion des stocks de boissons, en particulier pour les cavistes, bars, restaurants et distributeurs. Grâce à une interface intuitive, Winventory permet de suivre les entrées et sorties de stock, gérer les fournisseurs, et optimiser l'approvisionnement en fonction des ventes et des besoins.

Fonctionnalités Principales

Gestion des Stocks

- Visualisation en temps réel des articles en stock.
- Alertes automatiques pour les produits à faible quantité.

Gestion des Commandes et Approvisionnements

- Consultation des commandes en attente.
- Mise à jour des réceptions de marchandises avec validation des quantités.

Gestion des Fournisseurs

- Ajout des fournisseurs via une interface dédiée.

Interface Web Moderne et Ergonomique

- Navigation fluide et design optimisé pour une expérience utilisateur agréable.
- Thème clair/sombre pour un confort visuel personnalisé.
- Menu interactif avec accès rapide aux différentes sections.

Technologie

- API RESTful basée sur **Javalin** et **PostgreSQL**.
- Déploiement avec **Docker & Traefik** pour une infrastructure robuste et scalable.

Objectif

Winventory vise à **simplifier** la gestion des stocks de boissons en offrant un suivi précis et une visibilité accrue sur les stocks, les commandes et les fournisseurs.

Que vous soyez un **gérant de bar**, un **responsable de stock**, ou un **caviste**, **Winventory** vous aide à éviter les ruptures de stock et à optimiser vos commandes pour une gestion plus efficace.

Gérez vos stocks intelligemment avec Winventory !

Compilation, édition et lancement du projet

Note : Il vous faudra Java temurin 21 et Maven installés sur votre machine pour compiler et lancer ce projet.

Vous pouvez les installer facilement grâce à [ce guide](#)

Compiler et lancer le projet en direct sur la machine hôte

Note : pour run le projet en local, il faut avoir une base de données PostgreSQL avec les réglages suivants (N'oubliez

pas de mettre à jour le mot de passe dans le fichier `src/main/java/ch/heigvd/dai/Main.java`):

- **Nom de la Base de données** : `winventory`
- **Nom d'utilisateur** : `postgres`
- **Mot de passe** : `WhateverYouWant`
- **Port** : `5432`

Le script SQL pour créer et remplir la base de données avec des données de base se trouve dans le dossier `db-scripts`.

Vous devez les exécuter dans l'ordre pour vous assurer que l'application fonctionne correctement et que les données sont correctement insérées.

Une fois cela fait, vous pouvez suivre les étapes suivantes pour compiler et lancer le projet en local :

1. **Cloner le dépôt** : `git clone git@github.com:La-Kirby-Team/BDR-Project.git`
2. **(Optionnel) Éditions** : Faites les modifications nécessaires dans le code source.
3. **Compiler le projet** : `mvn clean package`
4. **Lancer l'application** : `java -jar target/Winventory-0.9.jar`

Lancer le Projet avec Docker compose (DB incluse)

Note : pour run le projet avec Docker, il faut avoir [Docker](#) et [Docker-compose](#) installés sur votre machine.

1. **Cloner le dépôt** : `git clone git@github.com:La-Kirby-Team/BDR-Project.git`
2. **(Optionnel) Éditions** : Faites les modifications nécessaires dans le code source.
3. **Compiler le projet** : `mvn clean package`
4. **Construire l'image Docker** : `docker build -t winventory .`
5. **Lancer les conteneurs Docker (Vérifiez bien que votre terminal se trouve dans le dossier racine du projet) :**
`docker compose up`
6. **(Optionnel) Démarrer uniquement le serveur web :**
`docker run --rm --name winventory_web -p 80:8080 winventory:latest` (Assurez-vous qu'une base de données PostgreSQL soit accessible depuis le conteneur Docker)

Une fois cela fait, votre serveur web devrait être accessible à l'adresse `http://localhost` si vous avez lancé le serveur web en local, sinon, vous pouvez accéder à l'application web via l'adresse que vous devez changer dans le `docker-compose.yml` (les règles de redirections de Traefik).

Publier l'Application avec Docker

Pour publier une image Docker sur ghcr.io, il vous faudra un compte GitHub et un token d'authentification [cliquez ici](#) pour la marche à suivre en anglais pour en créer / récupérer un (Chapitre Authenticating with a personal access token (classic)).

1. **Cloner le dépôt** : `git clone git@github.com:La-Kirby-Team/BDR-Project.git`
2. **(Optionnel) Éditions** : Faites les modifications nécessaires dans le code source.
3. **Compiler le projet** : `mvn clean package`

4. **Construire l'image Docker** : `docker build -t wininventory .`

5. **Se connecter à GitHub Container Registry** : `docker login ghcr.io -u`
`VotrePseudoGithubICI` puis entrez votre token
d'authentification.

6. **Taguer l'image pour le dépôt Docker Hub** : `docker tag wininventory`
`ghcr.io/VotrePseudoGithubICI/wininventory:latest`

7. **Pousser l'image vers Docker Hub** : `docker push`
`ghcr.io/VotrePseudoGithubICI/wininventory`

Assurez-vous de remplacer `VotrePseudoGithubICI` par votre nom d'utilisateur GitHub.

Une fois l'image publiée, vous pouvez la déployer sur n'importe quel serveur en utilisant Docker :

1. **Tirer l'image depuis Docker Hub** : `docker pull`
`ghcr.io/VotrePseudoGithubICI/wininventory:latest`

2. **Lancer un conteneur avec l'image** : `docker run --rm --name wininventory_web -p`
`80:8080 wininventory:latest`

Assurez vous qu'une base de données PostgreSQL soit accessible depuis le conteneur Docker, le `docker-compose.yml` qui est
fourni vous permettra de lancer le projet ainsi que sa base de données.
Pour faire fonctionner le docker-compose, il suffit de lancer la commande `docker-compose up`
dans le dossier racine du
projet.

Cela permettra de déployer l'application et sa base de données sur votre serveur.



Documentation de l'API

L'API RESTful de Winventory permet d'interagir avec les différentes fonctionnalités de l'application.
Voici un aperçu
des principales routes disponibles :



Authentification

- `POST /api/login` : Authentification de l'utilisateur.
- `POST /api/logout` : Déconnexion de l'utilisateur.
- `PUT /api/register` : Inscription d'un nouvel utilisateur.



Vue des Stocks

- `GET /api/stock` : Récupérer la liste des articles en stock.



Gestion des Commandes

- `GET /api/orders-waiting` : Récupérer la liste des commandes en attente (pas encore livrées).
- `PUT /api/orders-confirm` : Confirmation de la livraison d'une commande.

Gestion des Fournisseurs

- `GET /api/providers` : Récupérer la liste des fournisseurs.
- `POST /api/providers` : Ajouter un nouveau fournisseur.
- `DELETE /api/providers/{id}` : Supprimer le fournisseur avec l'ID spécifié.

Toutes les routes nécessitent une authentification préalable via le endpoint `/api/auth/login`.

Pour plus de détails,

cliquez [ici](#)

Utilisation de l'application web avec cURL

Voici quelques exemples de commandes cURL pour interagir avec l'API RESTful de Winventory.

Authentification

Requête :

```
curl -X POST http://localhost/api/login -H "Content-Type: application/json" -d '{"username":"utilisateur1","password":"motdepasse123"}'
```

Réponse :

```
{
  "message": "Logged in successfully"
}
```

Requête :

```
curl -X POST http://localhost/api/logout -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

Réponse :

```
{
  "message": "Logged out successfully"
}
```

Requête :

```
curl -X PUT http://localhost/api/register -H "Content-Type: application/json" -d '{"username":"nouvelUtilisateur","password":"motdepasse456","email":"email@example.com"}'
```

Réponse :

```
{
  "message": "User registered successfully"
}
```



Vue des Stocks

Requête :

```
curl -X GET http://localhost/api/stock
```

Réponse :

```
[
  [
    "1",
    "Vin Rouge",
    "750",
    "Bouteille",
    "20"
  ],
  [
    "2",
    "Vin Blanc",
    "500",
    "Bouteille",
    "5"
  ],
  [
    "3",
    "Champagne",
    "750",
    "Bouteille",
    "2"
  ]
]
```



Gestion des Commandes

Requête :

```
curl -X GET http://localhost/api/orders-waiting
```

Réponse :

```
[
  {
    "produit": "Vin Rouge",
    "quantite": 50,
    "dateCommande": "2023-10-01",
    "joursDepuisCommande": 10,
    "mouvementStockId": 1
  },
  {
    "produit": "Vin Blanc",
    "quantite": 30,
    "dateCommande": "2023-10-05",
    "joursDepuisCommande": 6,
  }
]
```

```
"mouvementStockId": 2
}
]
```

Requête :

```
curl -X PUT http://localhost/api/orders-confirm -H "Content-Type: application/json" -d '{"id":1,"date":"2023-10-11","quantite":50}'
```

Réponse :

```
{
  "message": "Commande confirmée avec succès."
}
```

Gestion des Fournisseurs

Requête :

```
curl -X GET http://localhost/api/providers
```

Réponse :

```
[
  {
    "id": 1,
    "nom": "Fournisseur A",
    "adresse": "123 Rue Principale",
    "numeroTelephone": "+41 78 123 45 67"
  },
  {
    "id": 2,
    "nom": "Fournisseur B",
    "adresse": "456 Avenue Secondaire",
    "numeroTelephone": "+41 78 765 43 21"
  }
]
```

Requête :

```
curl -X POST http://localhost/api/providers -H "Content-Type: application/json" -d '{"name":"Fournisseur C","address":"789 Boulevard Tertiaire","phone":"+41 78 987 65 43"}'
```

Réponse :

```
{
  "message": "Fournisseur ajouté avec succès."
}
```

Requête :

```
curl -X DELETE http://localhost/api/providers/1"
```

Réponse :

```
{  
  "message": "Fournisseur supprimé avec succès."  
}
```

Installation et Configuration d'une Machine Virtuelle

Ce guide vous aidera à installer et configurer Winventory, mais nous partons du principe que vous possédez déjà une machine virtuelle avec une distribution Linux installée. Si vous n'avez pas encore de machine virtuelle, vous pouvez suivre ce [guide](#) pour installer Ubuntu 20.04 LTS sur VirtualBox. Vous pouvez également utiliser un fournisseur de cloud comme AWS, Azure ou Google Cloud pour créer une machine virtuelle.



Installation de Docker et Docker Compose

1. Mettre à jour l'index des paquets :

```
sudo apt update
```

2. Installer les paquets permettant à apt d'utiliser un dépôt via HTTPS :

```
sudo apt install apt-transport-https ca-certificates curl software-properties-common
```

3. Ajouter la clé GPG officielle de Docker :

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg
```

4. Configurer le dépôt stable :

```
echo "deb [arch=amd64 signed-by=/usr/share/keyrings/docker-archive-keyring.gpg]  
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee  
/etc/apt/sources.list.d/docker.list > /dev/null
```

5. Mettre à jour l'index des paquets :

```
sudo apt update
```

6. Installer Docker :


```
sudo apt install docker-ce docker-ce-cli containerd.io
```

7. Vérifier que Docker est correctement installé :

```
sudo docker --version
```

8. Installer Docker Compose :

```
sudo apt install docker-compose
```

9. Vérifier que Docker Compose est correctement installé :

```
sudo docker-compose --version
```



Installation de SDKMAN!, Java SDK et Maven

SDKMAN! est un outil pratique pour gérer plusieurs versions de SDK sur votre machine. Voici comment l'installer :

1. Installer SDKMAN! :

```
curl -s "https://get.sdkman.io" | bash
```

2. **Suivre les instructions à l'écran** pour terminer l'installation. Vous devrez ouvrir un nouveau terminal ou exécuter la commande suivante pour initialiser SDKMAN! :

```
source "$HOME/.sdkman/bin/sdkman-init.sh"
```

3. Vérifier l'installation :

```
sdk version
```

4. Installer Java temurin 21 :

```
sdk install java 21.0.4-tem
```

5. Installer Maven :

```
sdk install maven
```

SDKMAN! est maintenant installé et configuré sur votre machine. Vous pouvez utiliser SDKMAN! pour gérer facilement les versions de Java et d'autres SDK nécessaires pour votre projet.

Configuration de la zone DNS pour accéder à l'application web avec DuckDNS

Pour configurer la zone DNS et accéder à votre application web via un domaine DuckDNS au lieu de son adresse ip publique, suivez les étapes ci-dessous :

1. Créer un compte DuckDNS

Rendez-vous sur [DuckDNS](#) et connectez-vous avec votre compte GitHub.

2. Ajouter un domaine

Cliquez sur le bouton `Add Domain` et choisissez un nom de domaine. Ce domaine sera utilisé pour accéder à votre application web.

3. Configurer les enregistrements DNS

Ajoutez les enregistrements DNS nécessaires pour pointer vers l'adresse IP de votre machine virtuelle.

Ajouter un enregistrement `A`

- **Nom de domaine :** `votre-domaine.duckdns.org`
- **Adresse IP :** `IP_de_votre_machine_virtuelle`

Ajouter un enregistrement `A` générique (wildcard)

- **Nom de domaine :** `*.votre-domaine.duckdns.org`
- **Adresse IP :** `IP_de_votre_machine_virtuelle`

4. Tester la résolution DNS

Depuis votre machine locale et votre machine virtuelle, testez la résolution DNS avec la commande suivante :

```
nslookup votre-domaine.duckdns.org
```

Vous devriez obtenir une réponse avec l'adresse IP de votre machine virtuelle.

5. Accéder à l'application web

Utilisez le domaine configuré pour accéder à votre application web. Par exemple, `http://votre-domaine.duckdns.org`.

En suivant ces étapes, vous pourrez configurer la zone DNS pour accéder à votre application web via DuckDNS.



Détails de l'API



Authentification

Endpoints

POST /api/login

- **Description:** Cet endpoint permet à un utilisateur de se connecter en fournissant ses identifiants.
- **Requête:** La requête doit contenir un objet JSON avec les informations suivantes :
 - **username:** Le nom d'utilisateur de l'utilisateur.
 - **password:** Le mot de passe de l'utilisateur.

```
{
  "username": "utilisateur1",
  "password": "motdepasse123"
}
```

- **Réponse:** La réponse contient un message de succès et un token JWT si les informations d'identification sont valides.

```
{
  "message": "Logged in successfully",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

POST /api/logout

- **Description:** Cet endpoint permet à un utilisateur de se déconnecter en fournissant un token d'authentification valide.
- **Requête:** Il n'y a pas de données spécifiques requises dans la requête, seul le cookie de la session est supprimé.
- **Réponse:** La réponse indique si la déconnexion a été effectuée avec succès.

```
{
  "message": "Logged out successfully"
}
```

PUT /api/register

- **Description:** Cet endpoint permet de créer un nouvel utilisateur en fournissant les informations nécessaires.
- **Requête:** La requête doit contenir un objet JSON avec les informations suivantes :
 - **username:** Le nom d'utilisateur du nouvel utilisateur.
 - **password:** Le mot de passe du nouvel utilisateur.

```
{
  "username": "nouvelUtilisateur",
  "password": "motdepasse456"
}
```

- **Réponse:** La réponse contient un message indiquant si l'utilisateur a été enregistré avec succès.

```
{
  "message": "User registered successfully"
}
```

Fonctionnement

- **Contrôleur:** `UserController`
 - Utilise Javalin pour définir les routes liées à l'authentification.
 - Gère les connexions, déconnexions et inscriptions des utilisateurs.
 - Vérifie les identifiants et génère des tokens JWT pour les connexions réussies.

Exemple de Réponses JSON

Connexion réussie:

```
{
  "message": "Logged in successfully",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Déconnexion réussie:

```
{
  "message": "Logged out successfully"
}
```

Inscription réussie:

```
{
  "message": "User registered successfully"
}
```



Vue des Stocks

Endpoints

GET /api/stock

- **Description:** Cet endpoint permet de récupérer la liste des articles en stock.
- **Requête:** Aucune donnée spécifique n'est requise dans la requête.

- **Réponse:** La réponse est une liste d'articles en stock, chaque article étant représenté par un tableau de chaînes de caractères. Chaque tableau contient les informations suivantes :
 - **ID:** L'identifiant unique de l'article.
 - **Nom du produit:** Le nom du produit.
 - **Volume:** Le volume de l'article.
 - **Recipient:** Le récipient de l'article.
 - **Quantité:** La quantité disponible de l'article.

GET /api/articles-lowQT

- **Description:** Cet endpoint permet de récupérer la liste des articles dont la quantité est faible.
- **Requête:** Aucune donnée spécifique n'est requise dans la requête.
- **Réponse:** La réponse est similaire à celle de l'endpoint `/api/stock`, mais ne contient que les articles dont la quantité est considérée comme faible.

Fonctionnement

- **Contrôleur:** `StockController`
 - Le contrôleur utilise Javalin pour définir les routes de l'API.
 - Il utilise une connexion à une base de données asynchrone pour exécuter des requêtes SQL et récupérer les données.
- **Requêtes SQL:**
 - `stockQuery.sql`: Cette requête SQL récupère les informations sur les articles en stock en joignant plusieurs tables (`Produit`, `Article`, `MouvementStock`) et en utilisant des fonctions de regroupement et de coalescence pour obtenir les données nécessaires.
 - `lowQTArticles.sql`: Cette requête SQL est similaire à `stockQuery.sql`, mais elle filtre les articles pour ne récupérer que ceux dont la quantité est faible.

Exemple de Réponse JSON

```
[
  [
    "1",
    "Vin Rouge",
    "750",
    "Bouteille",
    "20"
  ],
  [
    "2",
    "Vin Blanc",
    "500",
    "Bouteille",
    "5"
  ]
]
```

```
],  
[  
  "3",  
  "Champagne",  
  "750",  
  "Bouteille",  
  "2"  
]  
]
```

Gestion des Commandes

Endpoints

GET /api/orders-waiting

- **Description:** Cet endpoint permet de récupérer la liste des commandes en attente.
- **Requête:** Aucune donnée spécifique n'est requise dans la requête.
- **Réponse:** La réponse est une liste de commandes en attente, chaque commande étant représentée par un objet JSON contenant les informations suivantes :
 - **produit:** Le nom du produit commandé.
 - **quantite:** La quantité commandée.
 - **dateCommande:** La date de la commande.
 - **joursDepuisCommande:** Le nombre de jours écoulés depuis la commande.
 - **mouvementStockId:** L'identifiant unique du mouvement de stock associé à la commande.

PUT /api/orders-confirm

- **Description:** Cet endpoint permet de confirmer la réception d'une commande.
- **Requête:** La requête doit contenir un objet JSON avec les informations suivantes :
 - **id:** L'identifiant unique du mouvement de stock.
 - **date:** La date de réception de la commande.
 - **quantite:** La quantité reçue.
- **Réponse:** La réponse indique si la mise à jour de la commande a été effectuée avec succès ou si une erreur s'est produite.

Fonctionnement

- **Contrôleur:** `OrderController`
 - Le contrôleur utilise Javalin pour définir les routes de l'API.
 - Il utilise une connexion à une base de données asynchrone pour exécuter des requêtes SQL et récupérer les données.
- **Requêtes SQL:**

- `waitingOrders.sql`: Cette requête SQL récupère les informations sur les commandes en attente en joignant plusieurs tables (`Approvisionnement`, `MouvementStock`, `Article`, `Produit`) et en utilisant des fonctions de calcul pour obtenir les données nécessaires.

Exemple de Réponse JSON

```
[
  {
    "produit": "Vin Rouge",
    "quantite": 50,
    "dateCommande": "2023-10-01",
    "joursDepuisCommande": 10,
    "mouvementStockId": 1
  },
  {
    "produit": "Vin Blanc",
    "quantite": 30,
    "dateCommande": "2023-10-05",
    "joursDepuisCommande": 6,
    "mouvementStockId": 2
  }
]
```

Gestion des Fournisseurs

Endpoints

GET /api/providers

- **Description:** Cet endpoint permet de récupérer la liste des fournisseurs.
- **Requête:** Aucune donnée spécifique n'est requise dans la requête.
- **Réponse:** La réponse est une liste de fournisseurs, chaque fournisseur étant représenté par un objet JSON contenant les informations suivantes :
 - **id:** L'identifiant unique du fournisseur.
 - **nom:** Le nom du fournisseur.
 - **adresse:** L'adresse du fournisseur.
 - **numeroTelephone:** Le numéro de téléphone du fournisseur.

POST /api/providers

- **Description:** Cet endpoint permet d'ajouter un nouveau fournisseur.
- **Requête:** La requête doit contenir un objet JSON avec les informations suivantes :
 - **name:** Le nom du fournisseur.
 - **address:** L'adresse du fournisseur.
 - **phone:** Le numéro de téléphone du fournisseur.
- **Réponse:** La réponse indique si l'ajout du fournisseur a été effectué avec succès ou si une erreur s'est produite.

DELETE /api/providers/{id}

- **Description:** Cet endpoint permet de supprimer un fournisseur avec l'ID spécifié.
- **Requête:** Aucune donnée spécifique n'est requise dans la requête.
- **Réponse:** La réponse indique si la suppression du fournisseur a été effectuée avec succès ou si une erreur s'est produite.

Fonctionnement

- **Contrôleur:** `ProviderController`
 - Le contrôleur utilise Javalin pour définir les routes de l'API.
 - Il utilise une connexion à une base de données asynchrone pour exécuter des requêtes SQL et récupérer les données.
- **Requêtes SQL:**
 - `providerQuery.sql`: Cette requête SQL récupère les informations sur les fournisseurs en les triant par nom.
 - `providerNew.sql`: Cette requête SQL insère un nouveau fournisseur dans la base de données.
 - `deleteProviderQuery.sql`: Cette requête SQL supprime un fournisseur de la base de données en fonction de son ID.

Exemple de Réponse JSON

```
[
  {
    "id": 1,
    "nom": "Fournisseur A",
    "adresse": "123 Rue Principale",
    "numeroTelephone": "+41 78 123 45 67"
  },
  {
    "id": 2,
    "nom": "Fournisseur B",
    "adresse": "456 Avenue Secondaire",
    "numeroTelephone": "+41 78 765 43 21"
  }
]
```